Microsoft

GH-900

GitHub Foundations

GitHub Foundations Part 1 of 2

Module 1

Introduction to Git

Agenda

GitHub Foundations Part 1

1 Introduction to Git

2 Introduction to GitHub

3 Introduction to GitHub's products

4 Configure code scanning on GitHub

5 Introduction to GitHub Copilot

6 Code with GitHub Codespaces

7 Manage your work with GitHub Projects

8 Communicate effectively on GitHub using Markdown

GitHub Foundations Part 2

9 Contribute to an open-source project on GitHub

10 Manage an InnerSource program by using GitHub

11 Maintain a secure repository by using GitHub best practices

12 Introduction to GitHub administration

13 Authenticate and authorize user identities on GitHub

14 Manage repository changes by using pull requests on GitHub

15 Search and organize repository history by using GitHub

16 Using GitHub Copilot with Python

Introduction

Git is not just for software developers, but also for teams that write documentation and collaborate on other work.

By the end of this module, you'll be able to:

- Understand what version control is.
- Learn about distributed version control systems like Git.
- Recognize the differences between Git and GitHub and understand their roles in the software development lifecycle

For an overview of the exercises in this module, see the video Introduction to Git Recap.

What is version control?

A version control system (VCS) is a program or set of programs that tracks changes to a collection of files. VCS is also known as software configuration management (SCM) system.

With a VCS, you can:

- View changes made to your project, when, and who made them.
- Include a message with each change and an explain.
- Retrieve past versions of the entire project or of individual files.
- Create branches, where changes can be made experimentally. This allows several different sets of changes to be worked on at the same time. Later, you can merge the changes you want to keep back into the main branch.
- Attach a tag to a version—for example, to mark a new release.

VS Code

See the video Overview of version control.

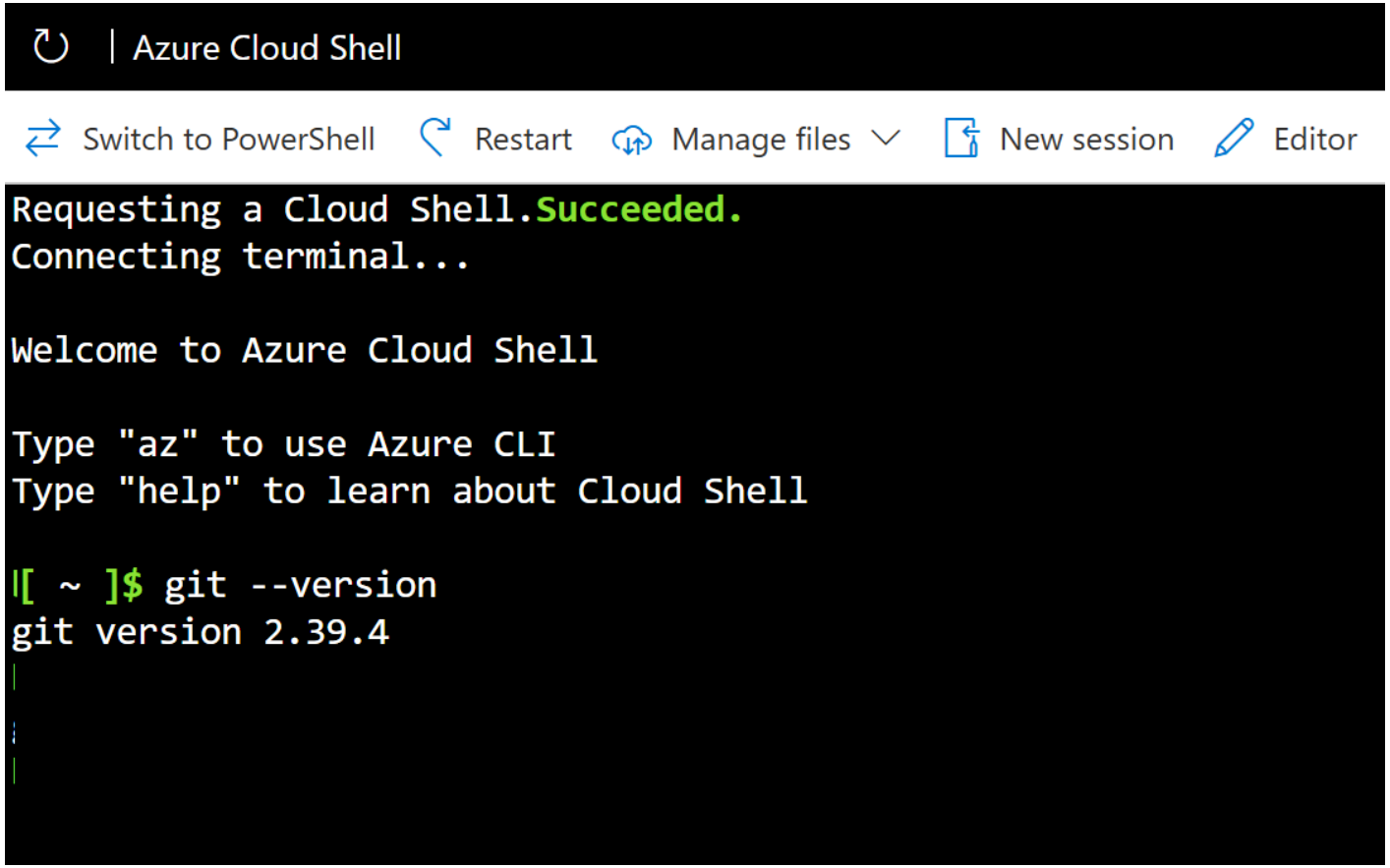
What is version control? (terminology)

- **Working tree:** The set of nested directories and files that contain the project.
- **Repository (repo):** The directory, located at the top level of a working tree, where Git keeps all the history and metadata for a project.
- **Hash:** A number produced by a hash function that represents the contents of a file or another object as a fixed number of digits.
- **Object:** A Git repo contains four types of objects:
  - *blob* object contains an ordinary file, *tree* object represents a directory, *commit* object represents a specific version of the working tree, and *tag* is a name attached to a commit.
- **Commit:** When used as a verb, commit means to make a commit object.
- **Branch:** A branch is a named series of linked commits. The default branch, which is created when you initialize a repository, is called main, often named master in Git.

Exercise - Try out Git

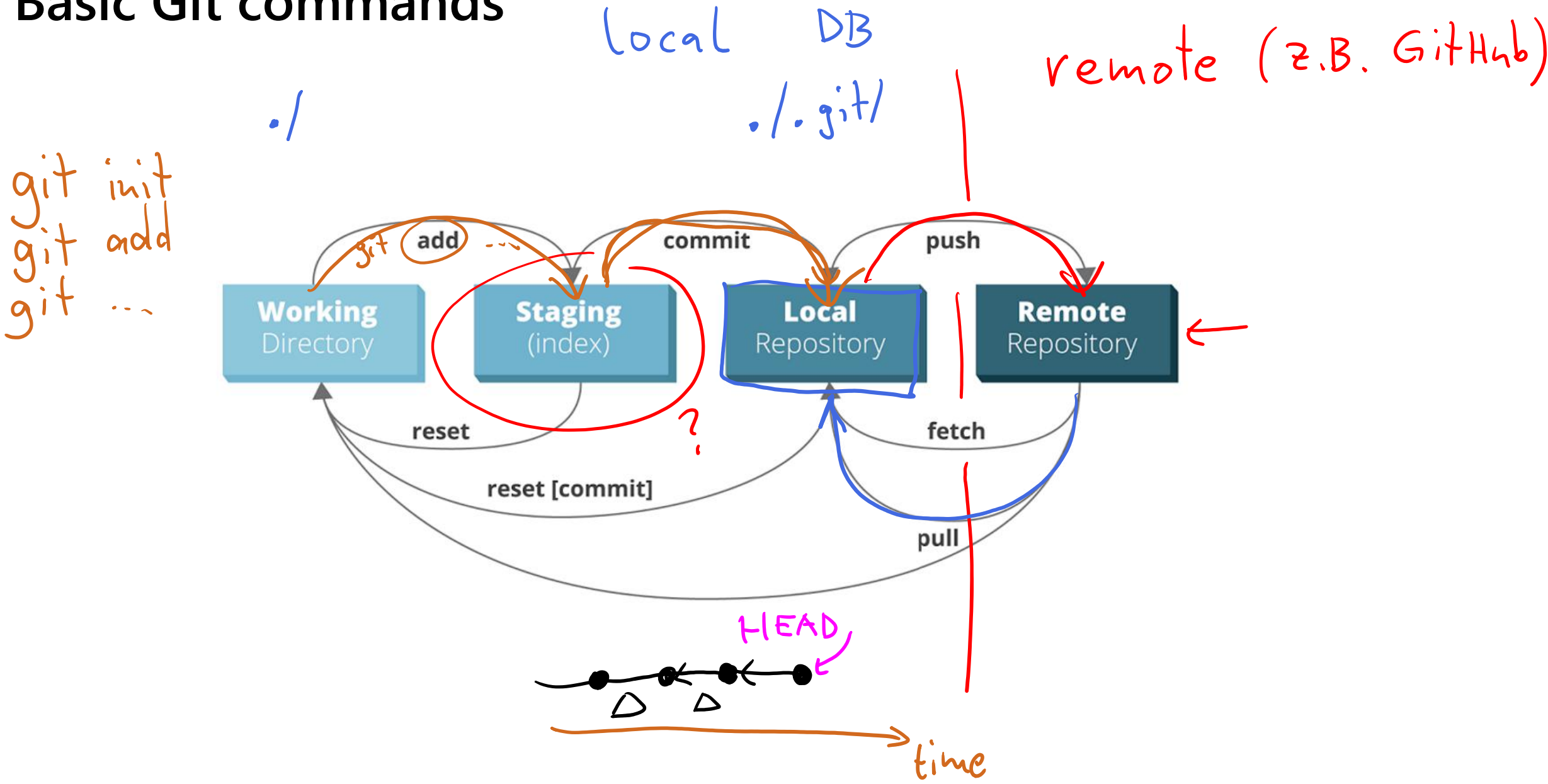
Before you can create your first repo, you must make sure that Git is installed and configured.

Git comes preinstalled with Azure Cloud Shell, so we can use Git in Cloud Shell to the right.



Visit Configure Git using the Azure Cloud Shell sandbox.

Basic Git commands



Basic Git commands

git status

`git status` displays the state of the working tree (and of the staging area—we'll talk more about the staging area soon). It lets you see which changes are currently being tracked by Git, so you can decide whether you want to ask Git to take another snapshot.

git add

`git add` is the command you use to tell Git to start keeping track of changes in certain files.

The technical term is *staging* these changes. You'll use git add to stage changes to prepare for a commit. All changes in files that have been added but not yet committed are stored in the *staging area*.

git commit

`git commit` has the same meaning as when you commit to a plan or commit a change to a database. As a verb, committing changes means you put a copy of the file or directory in the repository as a new version. As a noun, a commit is the small chunk of data that gives the changes you committed a unique identity.

Basic Git commands (cont.)

git log

The `git log` command allows you to see information about previous commits. Each commit has a message attached to it (a commit message), and the git log command prints information about the most recent commits, like their time stamp, the author, and a commit message. This command helps you keep track of what you've been doing and what changes have been saved.

git help

Use `git help` to easily get information about all the commands you've learned so far, and more.

Remember, each command comes with its *own* help page, too. You can find these help pages by typing `git <command> --help`.

For example, `git commit --help` brings up a page that tells you more about the git commit command and how to use it.

Summary

You learned:

- An overview of Version Control Systems (VCS)
- Git terminology.
- The differences between Git and GitHub.
- How to configure Git.
- Some basic Git commands.

If you'd like to dig deeper, Watch the Introduction to Git Recap video.

End of presentation